

PROMETHEUS & GRAFANA

HANDBOOK



The Complete Beginner's Guide to
Metrics, Monitoring & Observability
with Prometheus & Grafana



COLLECT
Real-time
Metrics



STORE
Time Series
Data



QUERY
Powerful
PromQL



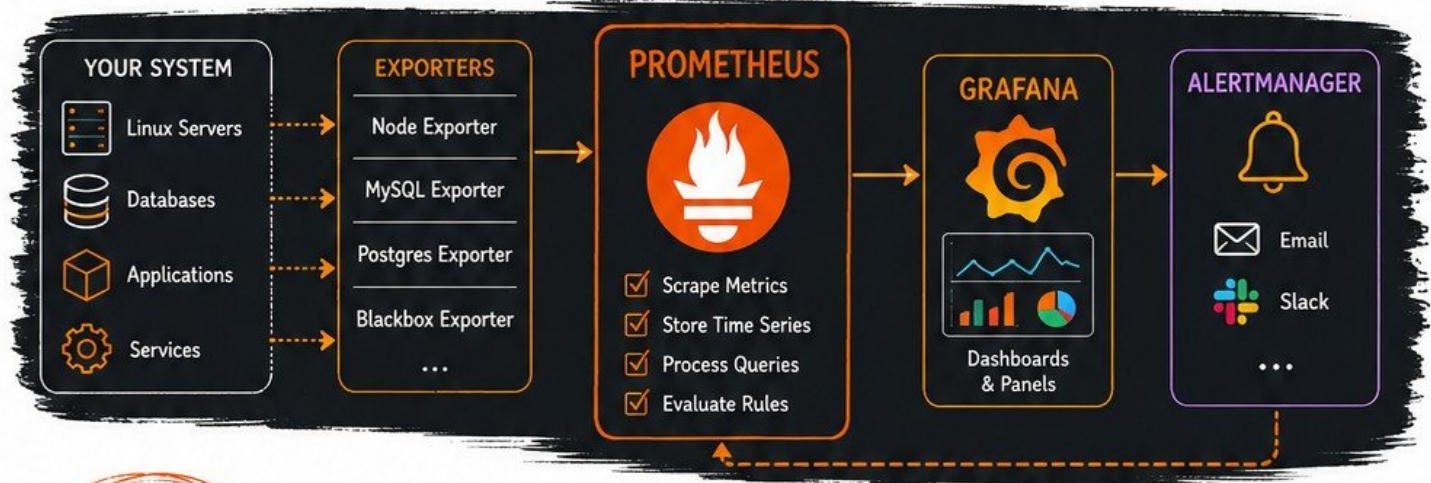
VISUALIZE
Beautiful
Dashboards



ALERT
Proactive
Alerting



OBSERVE
Understand
Your System



From Zero
to Production
Monitoring



**BEGINNER
FRIENDLY**
Clear explanations
with real examples



**HANDS-ON
PROJECTS**
Step-by-step labs
to build skills



**BEST
PRACTICES**
Production-ready
guidance



**INTERVIEW
READY**
Concepts & tips
for success

BUILD OBSERVABILITY. MASTER MONITORING. SHIP RELIABLE SYSTEMS.

1. WHY MONITORING MATTERS



LEARNING OBJECTIVES

- ✓ What monitoring is
- ✓ Why modern systems need monitoring
- ✓ Reactive vs proactive operations
- ✓ Real production example of a system outage

REMEMBER THIS

You can't fix
what you
can't see.

WHAT IS MONITORING?



Monitoring is the continuous process of collecting, analyzing, and visualizing data from systems, applications, and infrastructure to understand their health, performance, and availability.

WHY MODERN SYSTEMS NEED MONITORING?

- Modern systems are complex and distributed.
- Issues can occur at any layer.
- Users expect 24/7 availability.
- Small issues can become major outages.
- Monitoring helps you detect problems early before users are affected.

REACTIVE vs PROACTIVE OPERATIONS

REACTIVE (BAD)	PROACTIVE (GOOD)
✗ Issues found by users	✓ Issues found by monitoring
✗ Downtime & Outages	✓ High Availability
✗ Manual Troubleshooting	✓ Alerts before impact
✗ High Stress	✓ Less Stress
✗ Lost Trust	✓ Happy Users
✗ High Cost	✓ Lower Cost

REAL PRODUCTION EXAMPLE

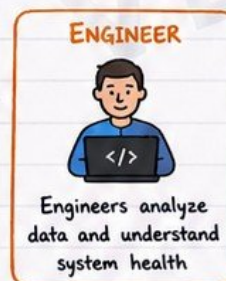


A payment API started timing out during peak traffic. Users faced payment failures. No monitoring was in place. The team noticed only after thousands of users complained. Revenue was lost and trust was damaged.

If monitoring was in place...

The team would have been alerted early and fixed the issue before users noticed.

THE MONITORING WORKFLOW



KEY TAKEAWAY



Monitoring is not optional. It is the foundation of reliability. It helps you detect problems early, reduce downtime, save money, and keep your users happy.

Be Proactive,
Not Reactive!



2. MONITORING vs LOGGING vs TRACING

MONITORING



Monitoring is the continuous collection of **METRICS** to understand the **HEALTH**, **PERFORMANCE**, and **AVAILABILITY** of systems.

Examples

- CPU usage
- Memory utilization
- Request rate
- Error rate
- Response time

★ Answers: "How is the system performing right now?"

LOGGING



Logging is the process of collecting, storing, and analyzing **LOGS** that record discrete events in a system.

Examples

- Error messages
- User logins
- Payment failed
- File not found
- Service started

★ Answers: "What happened in the system?"

DISTRIBUTED TRACING



Distributed Tracing tracks a request as it travels across multiple services to understand **LATENCY**, **BOTTLENECKS**, and **FAILURES**.

Examples

- Request flow across services
- Identify slow service
- See where errors occur
- Trace IDs for requests
- End-to-end latency

★ Answers: "Why did this request take so long?"

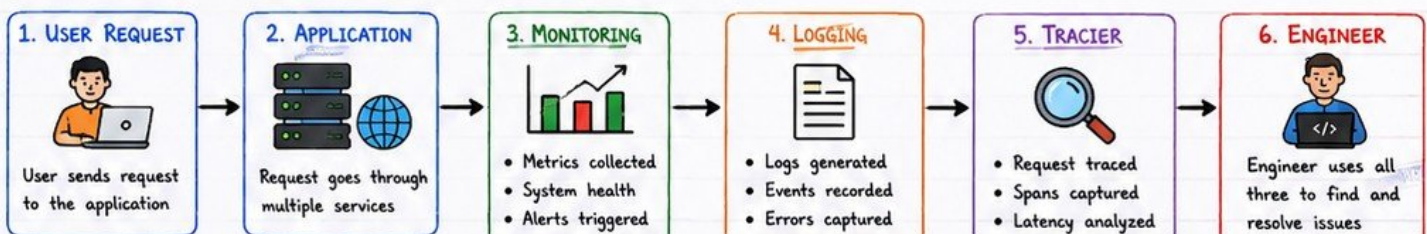
COMPARISON TABLE

ASPECT	MONITORING	LOGGING	DISTRIBUTED TRACING
What it collects	Metrics (numbers, counts, rates)	Logs (events, text, messages)	Traces (request flow across services)
Data type	Numeric time-series data	Textual event data	Span data with timing
Purpose	Know system health & performance	Know what happened	Know why it happened and where
Granularity	Aggregated over time	Individual events	Per-request / End-to-end
Storage	Time-series DB (e.g., Prometheus)	Log stores (e.g., Loki, ELK)	Trace stores (e.g., Tempo, Jaeger)
Best for	Alerts, dashboards, SLOs	Debugging, audit, compliance	Performance issues, bottlenecks
Retention	Short to medium (days/weeks)	Medium to long (days/months)	Short to medium (days/weeks)
Examples	CPU%, Memory, RPS, Errors	Error logs, Access logs, App logs	Request traces, Span details
Key Question	How is the system doing?	What happened?	Why did this happen?

WHEN TO USE WHAT?

- ✓ Use **MONITORING** when:
You want to track health, set alerts, or watch trends.
- ✓ Use **LOGGING** when:
You need details about events, errors, or user actions.
- ✓ Use **TRACING** when:
You need to find the root cause of slow requests across multiple services.

HOW ALL THREE WORK TOGETHER IN PRODUCTION



REAL EXAMPLE

A payment request fails.

Monitoring shows high error rate on payment service

Logs show "Database timeout" error

Tracing shows the request was slow in the database call

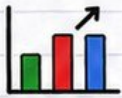
✓ Engineer fixes the DB issue and error rate returns to normal



3. THE THREE PILLARS OF OBSERVABILITY

Observability is the ability to understand the internal state of a system by examining its external outputs.

1. METRICS



Numerical measurements collected over time to understand the **HEALTH** and **PERFORMANCE** of a system.

Examples

- CPU usage
- Memory utilization
- Request rate
- Error rate
- Response time

★ Best for answering:
"How is the system doing?"

2. LOGS



Discrete events recorded by a system that provide detailed context about **WHAT HAPPENED**.

Examples

- Error messages
- User logins
- Payment failed
- File not found
- Service started

★ Best for answering:
"What happened?"

3. TRACES



Records a request as it travels across multiple services to show the **ENTIRE JOURNEY**.

Examples

- Request flow across services
- Find slow service
- Identify bottlenecks
- End-to-end latency
- Dependency breakdown

★ Best for answering:
"Why did it happen?"

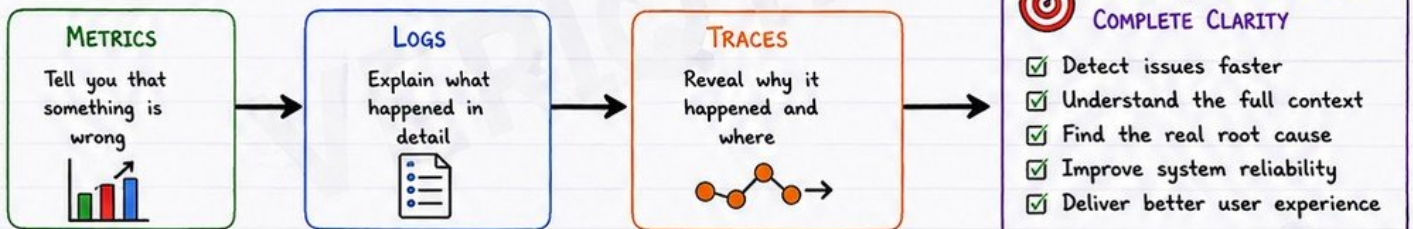
REMEMBER

Metrics show the symptoms.

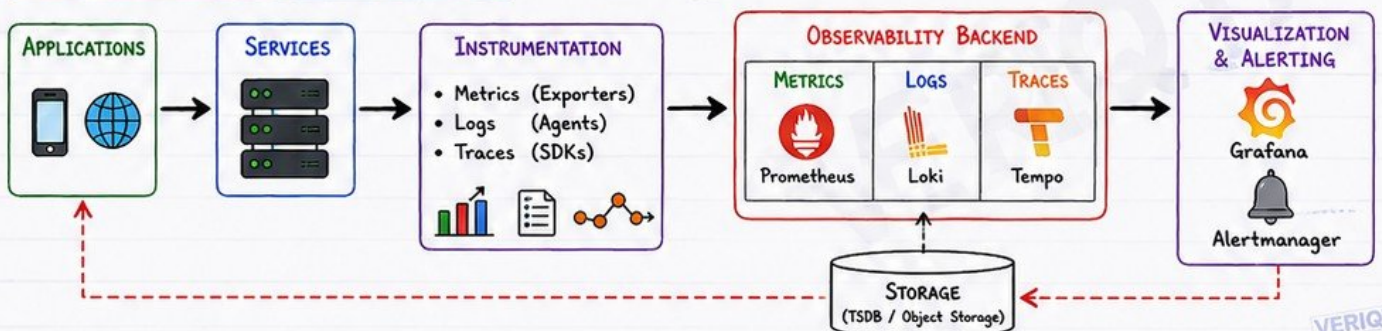
Logs show the details.

Traces show the root cause.

HOW THEY COMPLEMENT EACH OTHER



PRODUCTION ARCHITECTURE DIAGRAM



KEY TAKEAWAY

When you combine **METRICS**, **LOGS**, and **TRACES**, you move from guessing to knowing. This is the foundation of modern observability.



Prometheus

4. INTRODUCING PROMETHEUS

WHAT IS PROMETHEUS?

Prometheus is an **open-source** monitoring and alerting toolkit originally built at SoundCloud.

It collects **metrics** from configured targets at regular intervals, stores them in a **time-series** database, and provides a powerful query language (**PromQL**) to analyze the data.

REMEMBER

Prometheus is a monitoring system built for **reliability**, **scalability**, and **simplicity**.



HISTORY

- **2012:** Started at SoundCloud to monitor their large infrastructure.
- **2016:** Open sourced.
- **2017:** Joined CNCF incubation.
- **2018:** Graduated as a CNCF Graduated Project.

CNCF STATUS



Prometheus is a CNCF **Graduated** Project, which means it is production-ready, stable, and trusted by the cloud native community.

WHY COMPANIES USE PROMETHEUS

- ✓ Open Source and **Free**
- ✓ Highly scalable
- ✓ Powerful query language (**PromQL**)
- ✓ Great community and ecosystem
- ✓ Works well with Kubernetes
- ✓ Flexible and extensible (**Exporters**)
- ✓ Supports alerting natively (**Alertmanager**)
- ✓ Long-term storage integration (**Thanos**, **Mimir**)

PROBLEMS PROMETHEUS SOLVES

- ✓ Lack of visibility into system health
- ✓ Reactive troubleshooting
- ✓ No early detection of issues
- ✓ Poor alerting and noisy alerts
- ✓ No historical metrics for analysis
- ✓ Difficulty understanding performance
- ✓ Scaling monitoring across environments



WHEN NOT TO USE PROMETHEUS

- ✗ When you need log storage (use **Loki** / **Elasticsearch**)
- ✗ When you need full distributed tracing (use **Jaeger** / **Tempo**)
- ✗ When you need complex relational queries (use **SQL DB**)
- ✗ When you have extremely high cardinality metrics
- ✗ When you need real-time log search and analytics
- ✗ When retention requirements are extremely long (years without external storage)



PRODUCTION EXAMPLE

A payment processing platform experienced slow checkouts.



Outcome:

- Prometheus detected high latency in the database early.
- Alertmanager notified the engineer via Slack.
- The engineer identified a slow query and fixed it before users were heavily impacted.



INTERVIEW TIP

Be ready to explain why Prometheus uses a pull model and how it differs from push-based systems.



KEY TAKEAWAY

Prometheus helps you **detect problems early**, **understand performance**, and keep systems **reliable** as they scale.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

5. PROMETHEUS ARCHITECTURE

Prometheus follows a **pull-based model** to collect metrics from targets, stores them as **time series data**, and provides powerful querying and alerting.

REMEMBER

Prometheus collects **metrics**.
It does **NOT** receive metrics (except Pushgateway).

PROMETHEUS ARCHITECTURE DIAGRAM



PROMETHEUS SERVER

- The core component.
- Scrapes metrics from targets at configured intervals.
- Stores data in a time series database (TSDB).
- Processes queries (PromQL).
- Evaluates alerting & recording rules.



TARGETS

- Endpoints that expose metrics.
- Examples: servers, services, databases, APIs, applications.
- Prometheus **pulls** metrics from these endpoints.



EXPORTERS

- Expose metrics about systems or services in a format Prometheus understands.
- Bridge the gap between applications and Prometheus.
- Example: Node Exporter, MySQL Exporter, Blackbox Exporter.



ALERTMANAGER

- Handles alerts sent by Prometheus.
- Groups, deduplicates, and routes alerts to the right people.
- Supports silences, inhibitions, and integrations (Slack, Email, etc.).



GRAFANA

- Visualization & dashboarding tool.
- Connects to Prometheus to visualize metrics.
- Helps engineers explore, analyze, and understand system behavior.

SERVICE DISCOVERY



Finds Targets



PROMETHEUS SERVER

Service Discovery

Scrape Manager

TSDB
(Time Series DB)

Query Engine
(PromQL)

Alerting Engine

Sends Alerts



ALERTMANAGER

Grouping
Deduplication
Routing
Silences/Inhibition

Notify

NOTIFICATION CHANNELS

Email
Slack
PagerDuty
Webhook
...

TARGETS (Endpoints)



Pulls Metrics

Query (PromQL)

Visualize Metrics



EXPORTERS

- Node Exporter
- Blackbox Exporter
- MySQL Exporter
- Postgres Exporter
- Nginx Exporter
- cAdvisor
- Custom Exporter



DATA FLOW (STEP-BY-STEP)

- 1 Service Discovery finds available targets.
- 2 Prometheus Server **pulls** metrics from targets at regular intervals.
- 3 Metrics are stored in the Time Series Database.
- 4 Engineers query data using PromQL.
- 5 Alerting Engine evaluates rules and sends alerts to Alertmanager.
- 6 Alertmanager routes alerts to the right channels.
- 7 Grafana visualizes metrics for engineers.

KEY TAKEAWAY

Prometheus is the heart of your monitoring stack. It **collects**, **stores**, **queries**, and **alerts**. Everything else (Exporters, Alertmanager, Grafana) works together to give you full **observability**.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA



6. PROMETHEUS COMPONENTS

Prometheus is made up of multiple components that work together to collect, store, query, and alert on metrics.

REMEMBER

Each component has a specific role, but together they make Prometheus **powerful**.

1. TSDB (TIME SERIES DATABASE)



- Stores all the metrics as **time series data**.
- Optimized for fast writes and high cardinality.
- Uses a custom storage format with **compression, indexing, and retention**.
- Built on top of the local storage engine.

2. PROMQL (QUERY LANGUAGE)



- Powerful query language to select, filter, aggregate, and analyze metrics.
- Supports functions, operators, and advanced expressions.
- Used in Prometheus UI, Grafana, Alerting, and APIs.

3. SERVICE DISCOVERY



- Finds targets automatically.
- Supports multiple providers: **Kubernetes, EC2, Consul, Docker, Static Config**, etc.
- Ensures Prometheus always knows what to scrape.

4. SCRAPE MANAGER



- Responsible for **scraping** metrics from all targets.
- Runs scrapes at configured intervals.
- Handles timeouts, retries, and failures.

5. RULES ENGINE



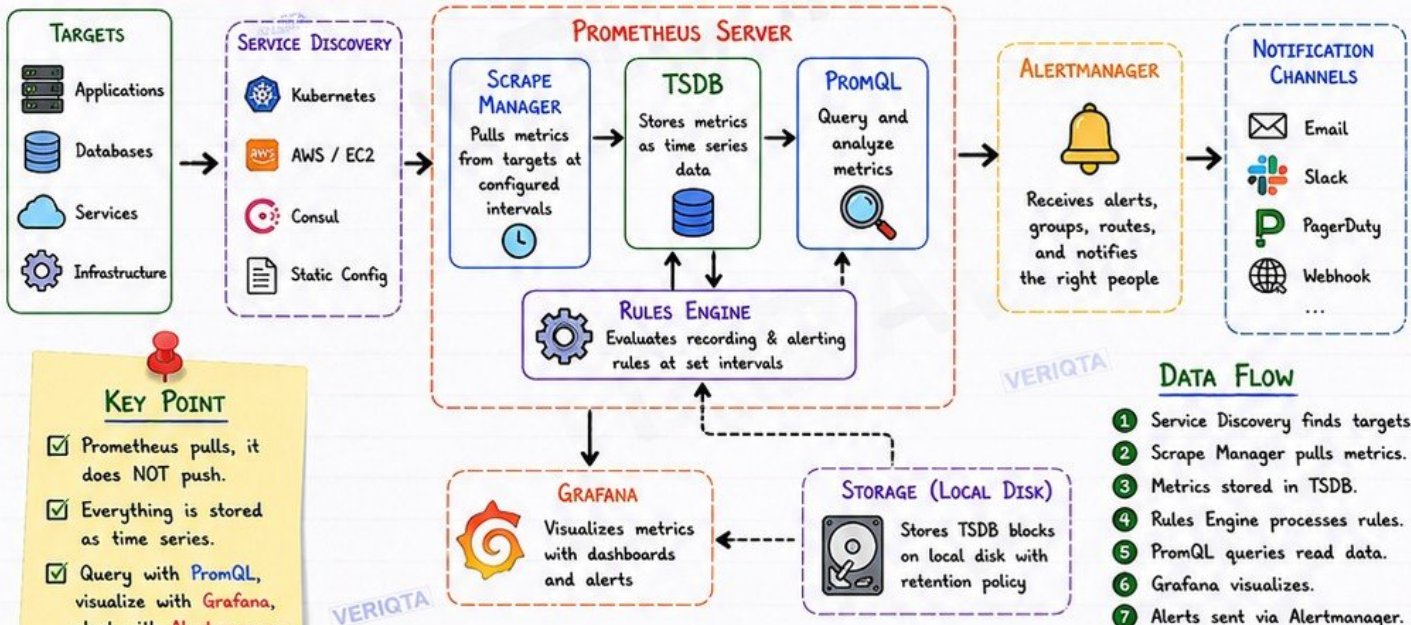
- Evaluates **recording rules** to generate new metrics.
- Evaluates **alerting rules** to detect issues.
- Runs on a schedule (every evaluation interval).

6. STORAGE (LOCAL STORAGE)



- Stores TSDB blocks on disk.
- Manages retention and compaction.
- Default retention is **15 days**, configurable.

HOW EVERY COMPONENT WORKS TOGETHER



KEY TAKEAWAY

Prometheus is a pipeline: **DISCOVER** → **SCRAPE** → **STORE** → **QUERY** → **ALERT** → **VISUALIZE**.
Each component has one job, but together they provide full **observability**.

REMEMBER

Prometheus pulls metrics.
Pushgateway is only for **short-lived** jobs.

7. PULL MODEL vs PUSH MODEL

Prometheus uses a **PULL MODEL** by default.

This is one of the core design decisions that makes Prometheus **simple, scalable, and reliable**.

WHY PULL?

- ✓ Simplicity
- ✓ Reliability
- ✓ Worker independence
- ✓ Centralized control

PULL MODEL (PROMETHEUS)

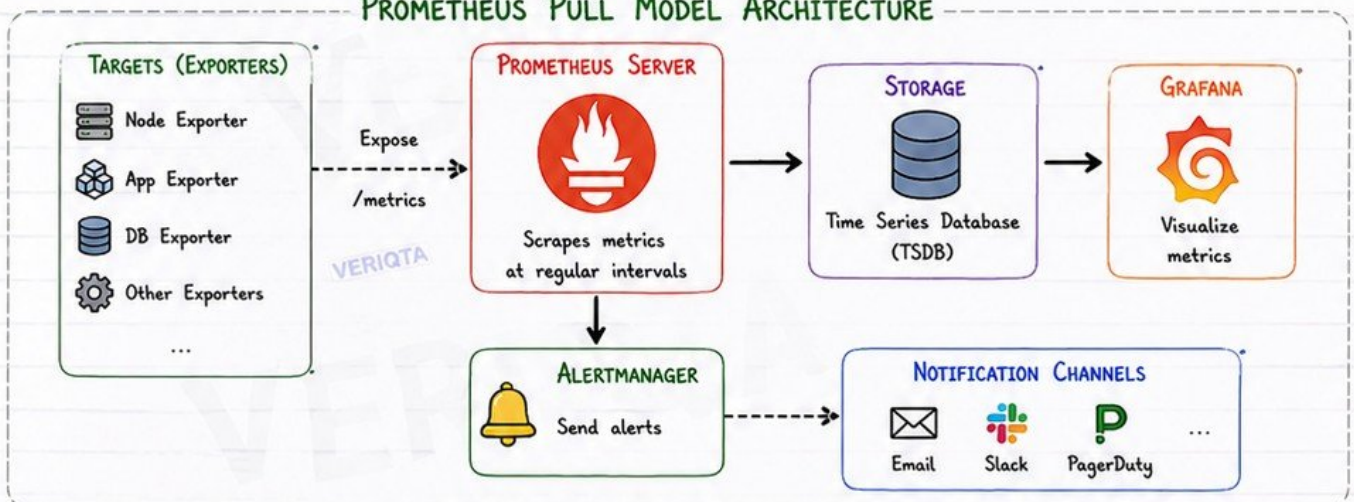
- Prometheus server periodically requests metrics from targets (exporters).
- Targets expose a /metrics endpoint.
- Prometheus scrapes metrics at regular intervals.
- Works great for long-lived services.



PUSH MODEL (PUSHGATEWAY)

- The **client** (application/job) pushes metrics to a gateway.
- Prometheus then pulls metrics from the gateway.
- Used for short-lived jobs or batch processes that don't exist long enough to be scraped.
- Pushgateway stores the metrics temporarily.

PROMETHEUS PULL MODEL ARCHITECTURE



PULL MODEL (PROMETHEUS)

Advantages

- ✓ Simple and easy to understand
- ✓ Centralized control (Prometheus decides when to scrape)
- ✓ No dependency on clients
- ✓ Better for security (targets don't need outbound access)
- ✓ More reliable for long-lived services

Disadvantages

- ✗ Does not work well for short-lived jobs
- ✗ Cannot push metrics immediately
- ✗ Requires targets to be available when scraped

PULL vs PUSH COMPARISON

	PULL MODEL (PROMETHEUS)	PUSH MODEL (PUSHGATEWAY)
Who initiates?	Prometheus (server)	Client (application/job)
Data flow	Prometheus pulls from /metrics endpoint	Client pushes to Pushgateway
Best for	Long-lived services	Short-lived / batch jobs
Storage	Prometheus TSDB	Pushgateway (temporary) → Prometheus TSDB
Reliability	High	Moderate (extra component)
Complexity	Simple	More components
Example Use	Web servers, DBs, Kubernetes, APIs	Cron jobs, CI/CD jobs, Data processing jobs

PUSHGATEWAY USE CASES

- Batch jobs
- Cron jobs
- CI/CD pipelines
- Scripts that run & exit
- Jobs that finish before Prometheus can scrape

★ PRO TIP

Use Pushgateway only when you have short-lived jobs. For everything else, let Prometheus pull.

REMEMBER

Prometheus is a
single binary with
no external
dependencies.

8. INSTALLING PROMETHEUS

Prometheus can run almost anywhere.
Here are the 3 most common installation methods.

YOU NEED

- ☒ Linux / Docker / K8s
- ☒ Network access
- ☒ Basic CLI knowledge



1. INSTALL ON LINUX

- 1 Download the latest release

```
$ wget https://github.com/prometheus/  
prometheus/releases/latest/download/  
prometheus-linux-amd64.tar.gz
```

- 2 Extract the binary

```
$ tar -xvf prometheus-linux-amd64.tar.gz  
$ cd prometheus-linux-amd64
```

- 3 Run Prometheus

```
$ ./prometheus --config.file=prometheus.yml
```

- 4 Access UI

→ Open <http://localhost:9090>



2. INSTALL WITH DOCKER

- 1 Pull the official image

```
$ docker pull prom/prometheus:latest
```

- 2 Run Prometheus container

```
$ docker run -d \  
-p 9090:9090 \  
--name prometheus \  
-v $(pwd)/prometheus.yml:/etc/prometheus/  
prometheus.yml \  
prom/prometheus:latest
```

- 3 Access UI

→ Open <http://localhost:9090>



3. INSTALL ON KUBERNETES (HELM)

- 1 Add Prometheus Helm repo

```
$ helm repo add prometheus-community \  
https://prometheus-community.github.io/helm-charts
```

- 2 Update repo

```
$ helm repo update
```

- 3 Install Prometheus

```
$ helm install prometheus prometheus-  
community/kube-prometheus-stack \  
--namespace monitoring --create-namespace
```

- 4 Access UI

• Port forward (for local access)

```
$ kubectl port-forward svc/prometheus-kube-  
prometheus-prometheus 9090:9090 -n monitoring
```

→ Open <http://localhost:9090>

VERIFY INSTALLATION

☒ 1. Check Service

- Linux (systemd)

```
$ systemctl status prometheus
```
- Docker

```
$ docker ps | grep prometheus
```
- Kubernetes

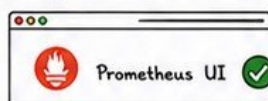
```
$ kubectl get pods -n monitoring
```



2. Check HTTP Endpoint

Open in browser:

<http://localhost:9090>



3. Check Targets

Go to: [Status](#) → [Targets](#)
All targets should be "UP"

TARGETS

• node-exporter:9100	UP
• blackbox-exporter:9115	UP
• cAdvisor:8080	UP

DIRECTORY STRUCTURE (LINUX INSTALL)

prometheus/	
prometheus	→ Main Prometheus binary
promtool	→ CLI tool for Prometheus
console_libraries/	→ UI libraries
consoles/	→ Web UI files
prometheus.yml	→ Main configuration file
rules/	→ Alerting & recording rules
data/	→ TSDB (Time Series Database)
01GZB3B... (blocks)	→ Time series blocks
wal/	→ Write Ahead Log

IMPORTANT CONFIG FILES

- prometheus.yml
Main config file for scraping & settings.
- rules/*.yml
Recording rules and alerting rules.
- alerts.yml
Alert definitions.

DEFAULT PORTS

	Prometheus	9090
	Alertmanager	9093
	Node Exporter	9100
	Blackbox Exporter	9115
	cAdvisor	8080



KEY TAKEAWAY

Install Prometheus using the method that fits your environment.
Once running, **verify targets** and **explore the UI**.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

9. UNDERSTANDING prometheus.yml



WHAT IS PROMETHEUS.YML?

It is the main configuration file for Prometheus.
It tells Prometheus what to scrape, how often to scrape, where to find targets, and what rules to evaluate.



KEY POINT

A good configuration file means **accurate metrics**, **efficient scraping**, and **reliable alerts**.

PROMETHEUS.YML - COMPLETE WALKTHROUGH

```
1 global:
2   scrape_interval: 15s
3   evaluation_interval: 15s
4   scrape_timeout: 10s
5   external_labels:
6     monitor: 'veriqta-monitor'
7
8 rule_files:
9   - "rules/*.yml"
10
11 scrape_configs:
12   - job_name: 'prometheus'
13     static_configs:
14       - targets: ['localhost:9090']
15
16   - job_name: 'node-exporter'
17     static_configs:
18       - targets: ['localhost:9100']
19
20   - job_name: 'blackbox-exporter'
21     metrics_path: /probe
22     params:
23       module: [http_2xx]
24     static_configs:
25       - targets: ['https://example.com']
26         labels:
27           instance: example.com
28           group: blackbox
```

1. GLOBAL

Contains global settings that affect the entire Prometheus server.

- **scrape_interval**: how often Prometheus scrapes targets.
- **evaluation_interval**: how often rules are evaluated.
- **scrape_timeout**: timeout for each scrape.
- **external_labels**: labels added to all metrics (useful for multi-env setups).

2. RULE_FILES

Path to rule files that define recording rules and alerting rules. Supports glob patterns.

3. SCRAPE_CONFIGS

The heart of the config. Defines all the jobs Prometheus will scrape.

- **job_name**: name of the job (shown in UI & used in queries).
- **static_configs / service discovery**: how Prometheus finds targets.
- **targets**: list of endpoints to scrape.

EVERY MAJOR SECTION EXPLAINED

1	global	Applies to the whole Prometheus server. Defines intervals, timeouts, and default labels.
2	evaluation_interval	How often Prometheus evaluates rules (recording & alerting). Default: 1m, but we set 15s for faster alerts.
3	scrape_interval	How often Prometheus scrapes targets. Lower interval = more data, higher load. Default: 1m, we set 15s.
4	rule_files	Where Prometheus looks for rule definitions. Helps keep configuration modular.
5	scrape_configs	Defines jobs and how to discover targets. Supports static, file, Kubernetes, Consul, EC2, and more.
6	job_name	Logical name for a scrape job. Used for grouping metrics in queries.

COMMON CONFIGURATION BEST PRACTICES

- ✓ Keep intervals reasonable (don't scrape too often).
- ✓ Use meaningful job names.
- ✓ Use external_labels for environment identification.
- ✓ Store rules in separate files.
- ✓ Use service discovery in production.
- ✓ Test configuration before reloading.



CHECK YOUR CONFIG

```
$ promtool check config prometheus.yml
```

This validates your configuration file and helps you catch syntax errors.

RELOAD PROMETHEUS AFTER CHANGES

```
$ curl -X POST http://localhost:9090/-/reload
```

This tells Prometheus to reload the config without restarting the service.



PRO TIP

Start simple. Add one job at a time and verify. Use the UI to confirm targets are UP.



NEXT PAGE PREVIEW

Next, we will explore Jobs and Targets in depth and how Prometheus discovers and scrapes them.





REMEMBER

Prometheus does not receive data. It **pulls** metrics from configured targets at regular intervals.

10. JOBS, TARGETS & SCRAPING

Prometheus works on a **pull model**. It discovers targets and scrapes metrics from them at a fixed interval, then stores them as **time series**.

1 JOB



- A job is a logical group of targets that share the same purpose.
- Example: `node-exporter`, `api-service`, `mysql`, `kubernetes-pods`
- Each job has its own scrape settings (interval, timeout, metrics path, etc).

2 TARGET



- A target is a specific instance that exposes metrics.
- Targets belong to a job.
- Example: `10.0.0.1:9100` (a `node-exporter` running on a server)

3 ENDPOINT



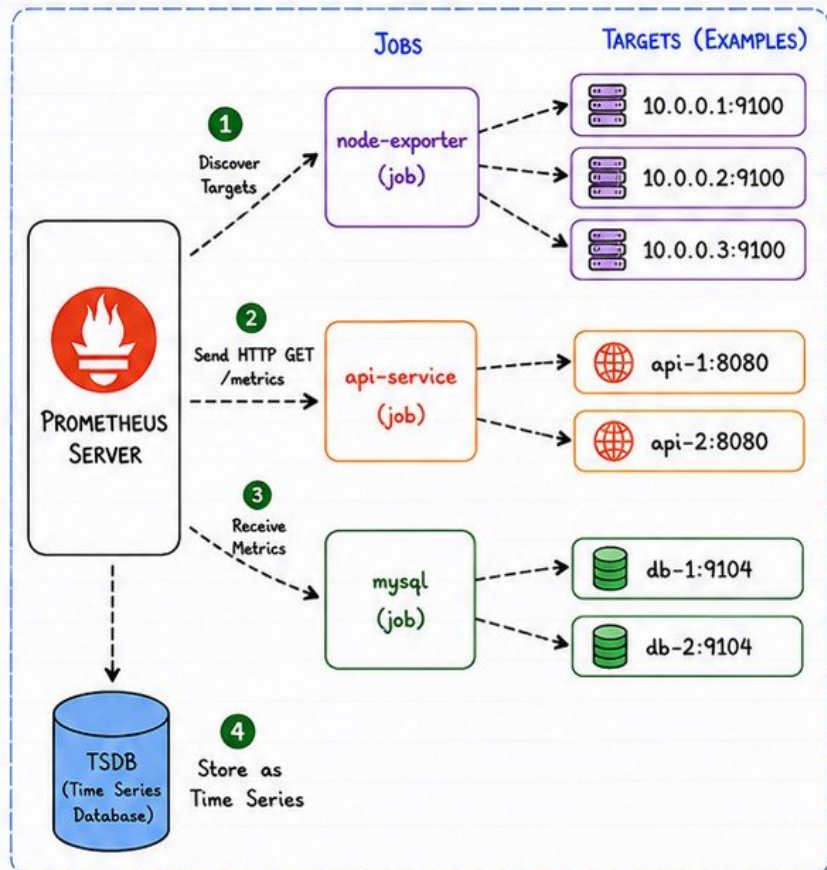
- The HTTP endpoint where metrics are available.
- Usually `/metrics`
- Example: `http://10.0.0.1:9100/metrics`

4 SCRAPING PROCESS



- Prometheus discovers targets for each job.
- At every `scrape_interval`, it sends an HTTP GET request to the target's `/metrics` endpoint.
- The target returns metrics in the Prometheus text format.
- Prometheus stores the metrics as **time series** in its TSDB.

HOW PROMETHEUS POLLS TARGETS



EXAMPLE CONFIG SNIPPET

```

scrape_configs:
  - job_name: 'node-exporter'
    static_configs:
      - targets: ['10.0.0.1:9100', '10.0.0.2:9100']
  - job_name: 'api-service'
    static_configs:
      - targets: ['api-1:8080', 'api-2:8080']
    metrics_path: /metrics
    scrape_interval: 15s
  
```



KEY TAKEAWAYS

- ✓ A Job groups multiple Targets.
- ✓ Each Target exposes metrics on an Endpoint.
- ✓ Prometheus is a Pull-based system.
- ✓ Metrics are scraped at regular intervals.
- ✓ Better scrape interval = more data (and more load).
- ✓ Make sure targets are **UP** and endpoints are reachable!



PRO TIP

Use meaningful job names and labels to identify your services easily.
Always monitor target health in: `Status` → `Targets`



REMEMBER

Service discovery helps Prometheus find targets **automatically** as your infrastructure changes.

11. SERVICE DISCOVERY

Service Discovery allows Prometheus to automatically find targets (instances) without manual configuration. It is key for **dynamic**, **scalable** environments.

BENEFITS

- ✓ No manual updates
- ✓ Scales automatically
- ✓ Reduces errors
- ✓ Handles ephemeral infrastructure
- ✓ Faster onboarding of new services

1. STATIC DISCOVERY

- Targets are defined manually in the configuration.
- Best for simple environments with few targets.
- You update the config file when targets change.

```
scrape_configs:
  - job_name: 'static-example'
    static_configs:
      - targets:
        - '10.0.0.1:9100'
        - '10.0.0.2:9100'
```

EXAMPLES

- Monitoring a single server
- Small apps with fixed infrastructure
- Blackbox probing of fixed endpoints

2. DYNAMIC DISCOVERY

- Targets are discovered automatically from an external system.
- Prometheus queries the service discovery endpoint at regular intervals.
- Best for cloud, containers, and orchestration platforms.

```
scrape_configs:
  - job_name: 'node-exporter'
    kubernetes_sd_configs:
      - role: node
    relabel_configs:
      - source_labels: [__address__]
        target_label: __address__
        replacement: $1:9100
```

EXAMPLES


 Kubernetes
(kubernetes_sd_configs)


 Docker
(docker_sd_configs)


 AWS EC2
(ec2_sd_configs)

... more

COMMON SERVICE DISCOVERY METHODS

METHOD	DESCRIPTION	USE CASE	EXAMPLE
Static	Manually defined targets	Small & stable environments	Virtual machines, on-prem servers
Kubernetes	Discover pods, services, nodes	K8s clusters	node-exporter, kube-state-metrics
Docker	Discover containers	Docker hosts	Containerized applications
AWS EC2	Discover EC2 instances	AWS infrastructure	Auto Scaling Groups, EC2 fleets
Consul	Discover services from Consul	Service mesh / Consul users	Microservices with Consul registry
File / DNS / HTTP	Discover targets from files, DNS records or HTTP endpoints	Specialized environments	Custom scripts, legacy systems

KUBERNETES DISCOVERY WORKFLOW

-  Prometheus queries Kubernetes API server
-  Kubernetes returns list of services/pods/nodes
-  Prometheus applies relabeling rules to filter / format targets
-  Targets are added to the scrape pool
-  Prometheus scrapes targets at configured intervals

PRODUCTION WORKFLOW



★ PRO TIP

Use **dynamic discovery** in production. It reduces manual work and ensures you never miss new or replaced instances.

💡 TIP

Combine **relabel_configs** to add job labels, drop unwanted targets, and ensure clean, meaningful metrics.



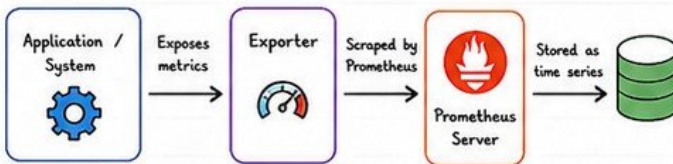
12. UNDERSTANDING METRICS

Prometheus is a **metrics-based** monitoring system. It collects, stores, and allows you to query **time series** data (metrics) about your systems.

WHAT IS A METRIC?

A metric is a numerical measurement of a system at a given point in time, often with labels that provide context.
Example: CPU usage 72% on server-01.

1 HOW METRICS ARE GENERATED



- Applications expose metrics via an HTTP endpoint (`/metrics`).
- Exporters collect metrics from the system and expose them.
- Prometheus **scrapes** the endpoint at regular intervals.
- Metrics are stored as **time series** in the TSDB.

2 METRIC NAMING CONVENTIONS

- Use lowercase letters, numbers, and underscores.
- Follow the pattern: `<namespace>_<subsystem>_<metric>_<unit>`
- Units should be included when applicable.
- Use labels for dimensions, not in the name.

✓ GOOD EXAMPLES

- `http_requests_total`
- `node_cpu_seconds_total`
- `process_resident_memory_bytes`
- `node_disk_io_time_seconds`

✗ BAD EXAMPLES

- `HTTPRequestsTotal`
- `node-cpu_seconds`
- `cpuUsagePercent`
- `http_requests`

3 COMMON METRIC TYPES

COUNTER

Monotonically increasing values that only go up (reset on restart).

Example:
`http_requests_total`

GAUGE

Represents a value that can go up and down.

Example:
`cpu_usage_percent`

HISTOGRAM

Tracks the distribution of values over configurable buckets.

Example:
`http_request_duration_seconds`

SUMMARY

Similar to histogram, but client-side quantiles.

Example:
`http_request_duration_seconds`

UNTYPED

Don't fit other types. Use sparingly.

Example:
`custom_metric_value`

4 REAL-WORLD METRIC EXAMPLES

CPU METRICS 	MEMORY METRICS 	DISK METRICS 	HTTP REQUEST METRICS
• <code>node_cpu_seconds_total</code> Total CPU time spent in each mode. Rate: <code>rate(node_cpu_seconds_total{mode="idle"}[5m])</code> Use: Calculate CPU usage percentage.	• <code>node_memory_MemTotal_bytes</code> Total physical memory. • <code>node_memory_MemAvailable_bytes</code> Available memory. • Use: Calculate memory usage.	• <code>node_filesystem_size_bytes</code> Filesystem total size. • <code>node_filesystem_free_bytes</code> Free space. • Use: Monitor disk usage.	• <code>http_requests_total</code> Total number of HTTP requests. • <code>http_request_duration_seconds</code> Request duration (histogram). • Use: Monitor traffic & latency.
CPU USAGE % $100 - \text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5m])) \cdot 100$	MEMORY USAGE % $100 - (\text{node_memory_MemAvailable_bytes} / \text{node_memory_MemTotal_bytes}) \cdot 100$	DISK USAGE % $100 - (\text{node_filesystem_free_bytes} / \text{node_filesystem_size_bytes}) \cdot 100$	REQUEST RATE <code>rate(http_requests_total[5m])</code> P95 LATENCY <code>histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))</code>

5 KEY TAKEAWAYS



- ✓ Metrics give visibility into system behavior.
- ✓ Use consistent naming and labels.
- ✓ Choose the right metric type for the job.
- ✓ Monitor what matters: CPU, Memory, Disk, and Request metrics are your foundation.
- ✓ Good metrics = Good observability = **Happy on-call engineer!**

13. UNDERSTANDING TIME SERIES

REMEMBER

Prometheus is a **time series** database (TSDB). Everything is a time series!

WHAT IS A TIME SERIES?

A **time series** is a sequence of data points indexed in **time order**. Prometheus stores all metrics as **time series**.

★ KEY CONCEPT

A metric is identified uniquely by:

Metric Name + Labels

Each sample has:

Timestamp + Value

1 CORE BUILDING BLOCKS OF A TIME SERIES

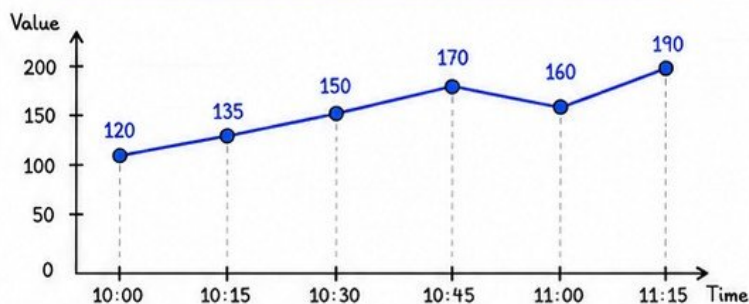
	TIMESTAMP	The exact point in time when the sample was recorded. Stored in Unix epoch (seconds).
	SAMPLE	A single data point captured at a specific timestamp.
	VALUE	The numerical value of the metric at that timestamp. (float64)
	LABELS	Key-value pairs that provide context and uniquely identify the time series. Example: {instance="server-01", job="node-exporter"}
	RETENTION	How long Prometheus keeps historical data before deleting it. Controlled by <code>--storage.tsdb.retention.time</code>
	TSDB	Time Series Database. Prometheus stores all time series in its local TSDB.

2 TIME SERIES VISUALIZED

Example Metric: **node_cpu_seconds_total**

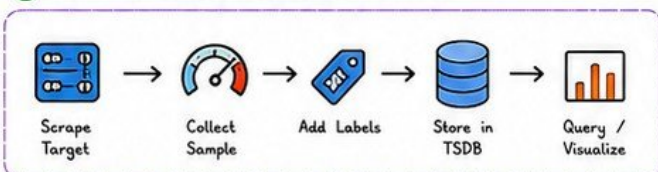
Labels:

{instance="server-01", job="node-exporter", mode="idle"}



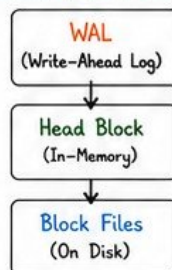
TIMESTAMP (UTC)	VALUE	LABELS
10:00:00	120	{instance="server-01", job="node-exporter", mode="idle"}
10:15:00	135	{instance="server-01", job="node-exporter", mode="idle"}
10:30:00	150	{instance="server-01", job="node-exporter", mode="idle"}
10:45:00	170	{instance="server-01", job="node-exporter", mode="idle"}
11:00:00	160	{instance="server-01", job="node-exporter", mode="idle"}
11:15:00	190	{instance="server-01", job="node-exporter", mode="idle"}

3 HOW PROMETHEUS STORES DATA (TSDB)



- Prometheus receives samples from targets at regular intervals.
- Each sample = **metric name + labels + timestamp + value**
- Samples are appended to the TSDB.
- Data is stored on disk in efficient blocks (chunked & compressed).
- Old data is removed automatically based on retention policy.

4 TSDB DETAILS



- **WAL:** Every sample is first written to the Write-Ahead Log for durability.
- **Head Block:** Recent samples kept in memory (fast reads).
- **Block Files:** Periodically written to disk as immutable blocks.
- **Index:** Maps series and timestamps to the block data.
- **Compression:** Reduces storage using delta + XOR encoding.

5 RETENTION & STORAGE

- Default retention: 15 days
- Change with: `--storage.tsdb.retention.time=30d`
- After retention period, old blocks are deleted.
- Plan disk capacity: ~2GB per million samples per day

Example:

If you scrape 1000 metrics every 15s
→ ~5.76 million samples per day
→ ~11.5 GB for 15 days

★ PRO TIP

Keep retention long enough for debugging incidents, but not too long to waste disk. Monitor disk usage & plan capacity!



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

REMEMBER

Use the right metric type for the job.
Wrong type =
Wrong insights!

14. PROMETHEUS METRIC TYPES

★ KEY POINT

All metric types are identified by a metric name + labels and are stored as time series (timestamp + value).

Prometheus supports 4 core metric types. Choosing the right one is critical for accurate monitoring and efficient storage.

1 COUNTER



- A monotonically increasing value that only goes up (or resets to 0 on restart).
- Use for values that represent a count of events.
- Examples: requests, errors, jobs processed.

PRODUCTION EXAMPLES



- http_requests_total
- node_cpu_seconds_total
- kafka_messages_in_total
- app_errors_total

2 GAUGE



- A value that can go up and down.
- Use for measurements that fluctuate.
- Examples: temperature, CPU usage, memory usage.

PRODUCTION EXAMPLES



- node_memory_MemAvailable_bytes
- node_load1
- node_filesystem_free_bytes
- process_resident_memory_bytes

3 HISTOGRAM



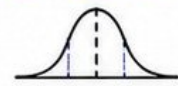
- Samples observations and counts them in configurable buckets.
- Use for measuring distributions (e.g., request duration).
- Enables calculation of quantiles (percentiles).

PRODUCTION EXAMPLES



- http_request_duration_seconds
- grpc_server_handling_seconds
- database_query_duration_seconds
- job_execution_time_seconds

4 SUMMARY



- Similar to histogram but calculates quantiles on the client side.
- Use when you need quantiles directly without querying.
- Less flexible than histogram across multiple instances.

PRODUCTION EXAMPLES

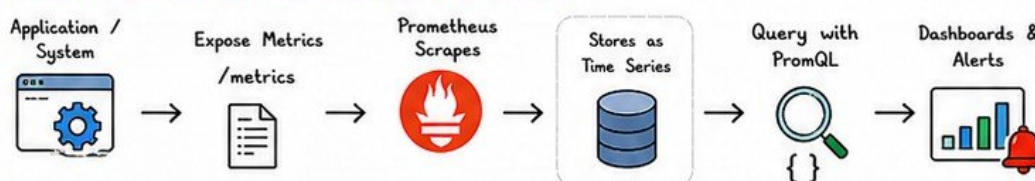


- http_request_duration_seconds
- rpc_client_latency_seconds
- api_response_time_seconds
- request_size_bytes_summary

METRIC TYPES COMPARISON

FEATURE	1 COUNTER	2 GAUGE	3 HISTOGRAM	4 SUMMARY
Behavior	Monotonically increases (only up)	Can go up and down	Bucketed counts of observations	Tracks quantiles on the client side
Use Case	Counting events	Measuring current state	Measuring distributions (server side)	Measuring distributions (client side)
Examples	Requests, Errors, Jobs Processed	CPU usage, Memory, Temp, Disk free	Request latency, Query duration, Job duration	Latency percentiles, Response time
Quantiles	No	No	Yes (via PromQL)	Yes (built-in)
Storage Cost	Low	Low	High (many time series per metric)	Medium
Aggregation Across Instances	Easy (sum)	Depends (avg, max, min)	Possible with care (sum by, quantile)	Not accurate (not aggregatable)
Reset on Restart	Yes (resets to 0)	No	Yes (buckets reset)	Yes
Best For	Total counts over time	Current values	Accurate percentiles at different levels	Client-side percentiles (low latency overhead)

HOW METRIC TYPES FIT IN PRODUCTION MONITORING



💡 TIP

Use Counter for totals, Gauge for current state, Histogram for distributions, Summary for client-side percentiles.

REMEMBER

Great labels =
Powerful metrics.
Bad labels =
Cardinality explosion!

15. LABELS & BEST PRACTICES

Labels are **key-value pairs** that attach **context** to metrics.
They make metrics discoverable, filterable,
and incredibly powerful.

LABEL FORMAT

```
{label_name="label_value"}
```

Examples:

- job="node-exporter"
- instance="10.0.0.1:9100"
- env="production"
- region="us-east-1"

1 WHAT ARE LABELS?



- Labels are metadata attached to metrics.
- Each metric can have multiple labels.
- Used to identify, group, and filter metrics.
- Example metric with labels:

```
http_requests_total{job="api", instance="10.0.0.5:8080",  
method="GET", status="200"} 1024
```

↑ Metric Name ↑ Labels ↓ Value

2 SELECTORS (FILTERING WITH LABELS)



- PromQL uses label selectors to filter metrics.
- Types of selectors:

SELECTOR	EXAMPLE	MEANING
=	job="api"	Matches exactly
!=	env!="dev"	Does not match
~	instance=~"10\.0\.0\..*"	Regex match
!~	status!~"5..."	Regex does not match

Example Query: `rate(http_requests_total{job="api",status=~"2.."}[5m])`

3 CARDINALITY



- Cardinality = total number of unique time series.
- High cardinality = high memory usage & slow queries.
- Main causes:
 - Using dynamic values as labels (user_id, session_id)
 - High number of unique label combinations
 - Unbounded label values



Keep cardinality under control.
Rule of thumb: < 10 million time series per server.

4 NAMING CONVENTIONS (BEST PRACTICES)

- Use clear, consistent, and meaningful names.
- Follow Prometheus naming best practices:
 - ✓ Use lowercase letters, numbers, and underscores.
 - ✓ Use _ as separator.
 - ✓ Start with a noun (what is measured).
 - ✓ Add unit as suffix (if applicable).
 - ✓ Use _total for counters.
 - ✓ Use _seconds for time values.

GOOD EXAMPLES

- ✓ node_cpu_seconds_total
- ✓ http_request_duration_seconds
- ✓ process_resident_memory_bytes
- ✓ kafka_messages_in_total

BAD EXAMPLES

- ✗ NodeCPUSecodsTotal
- ✗ HttpRequestDuration
- ✗ cpuUsagePercent
- ✗ requests

5 COMMON MISTAKES

MISTAKE	WHY IT'S A PROBLEM	EXAMPLE	SOLUTION
High Cardinality Labels	Blows up memory & storage	user_id, session_id, request_id	Avoid dynamic / unbounded labels
Inconsistent Label Names	Hard to query & aggregate	Env / environment / enviro	Use consistent naming
Missing Important Labels	Lack of context	No env, region, or service label	Add labels that provide context
Too Many Labels	Hard to read & slow queries	10+ labels per metric	Keep label count minimal
Changing Label Values	Breaks time series continuity	Changing instance ip frequently	Use stable identifiers
Using Labels for Data	Labels are for identity, not data	status_code="200", "500"	Use labels for dimensions, not values

6 LABELS BEST PRACTICES CHECKLIST



- ✓ Use labels to describe WHAT the metric is about, not the value.
- ✓ Keep label cardinality low and predictable.
- ✓ Prefer static, meaningful label values.
- ✓ Use environment, region, service, instance, version, and job labels.
- ✓ Review cardinality regularly (prometheus_tsdb_head_series()).
- ✓ Document your metric and label strategy.

7 INTERVIEW TIPS



- ★ Explain what labels are and how they help in filtering.
- ★ Discuss cardinality and why it matters.
- ★ Know the difference between labels and fields.
- ★ Be ready to write label selectors in PromQL.
- ★ Mention best practices and common mistakes.
- ★ Talk about real-world examples from production.



KEY TAKEAWAY

Good labels = accurate, scalable, and maintainable monitoring.
Bad labels = slow queries, high cost, and angry on-call engineers.

REAL-WORLD LABEL EXAMPLE

```
http_requests_total{  
  job="api-server",  
  instance="10.0.0.5:8080",  
  env="prod", region="us-east-1",  
  method="POST", status="200"} 5678
```



PRO TIP

Design your label strategy early.
A small mistake in labels can cost you big later!



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA



Exporters are the bridge between systems and Prometheus.

16. EXPORTERS

Exporters are small services that collect metrics from systems, applications, databases, or services that do not expose metrics in a Prometheus format.

★ WHY EXPORTERS EXIST

- Many systems do not expose metrics natively.
- Exporters translate system data into Prometheus format.
- They provide a consistent way to monitor anything.

1 NODE EXPORTER



- Exposes hardware and OS metrics.
- CPU, Memory, Disk, Network, Load, Filesystem, etc.

Example Use:
Monitor Linux/Unix servers.

Default Port: 9100

2 BLACKBOX EXPORTER



- Probes endpoints over HTTP, HTTPS, TCP, ICMP, DNS, etc.
- Used for uptime and availability monitoring.

Example Use:
Monitor website uptime, SSL expiry, API availability.

Default Port: 9115

3 MYSQL EXPORTER



- Exposes MySQL server metrics.
- Connections, Queries, Threads, Replication, InnoDB, etc.

Example Use:
Monitor MySQL performance and health.

Default Port: 9104

4 POSTGRES EXPORTER



- Exposes PostgreSQL metrics.
- Connections, Locks, Tuples, Transactions, Replication, etc.

Example Use:
Monitor PostgreSQL database performance.

Default Port: 9187

5 REDIS EXPORTER



- Exposes Redis metrics.
- Memory, Keys, Commands, Connections, Hit/Miss, etc.

Example Use:
Monitor Redis cache performance and usage.

Default Port: 9121

6 NGINX EXPORTER



- Exposes Nginx server metrics.
- Requests, Connections, Active Connections, Status, etc.

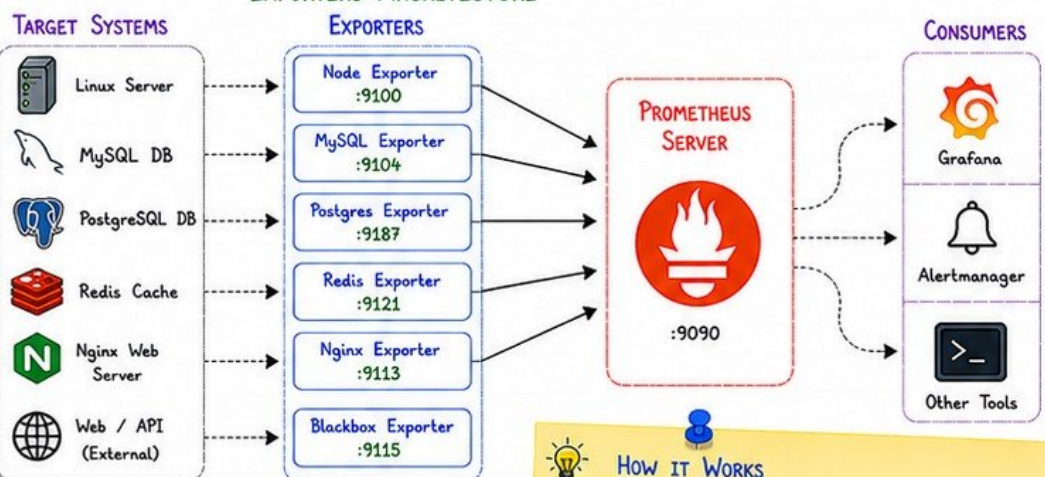
Example Use:
Monitor Nginx performance and traffic.

Default Port: 9113

★ EXPORTER BEST PRACTICES

- ✓ Run exporters close to the target system.
- ✓ Use least privilege accounts.
- ✓ Monitor exporters themselves.
- ✓ Use latest stable versions.
- ✓ Secure endpoints with firewalls.
- ✓ Document ports and configurations.

EXPORTERS ARCHITECTURE



★ HOW IT WORKS

- 1 Exporters collect metrics from target systems.
- 2 Exporters expose metrics in Prometheus format.
- 3 Prometheus scrapes exporters at regular intervals.
- 4 Data is stored, visualized, and alerted on.



Key TAKEAWAY: Exporters make anything monitorable. Pick the right exporter, run it reliably, and Prometheus will do the rest!

REMEMBER

This is the complete end-to-end journey of a metric from your application to an engineer.

17. END-TO-END PROMETHEUS DATA FLOW

THE COMPLETE JOURNEY

Metrics travel through multiple components before becoming insights that drive action.
Collect → Store → Query → Visualize → Alert → Act

★ KEY POINT

Every arrow in this flow represents a critical step in your monitoring pipeline.

1 APPLICATION

Your application, services, and infrastructure generate metrics (CPU, memory, requests, etc.).

2 EXPORTER

Exporter collects metrics from the target system and exposes them in Prometheus format.

3 PROMETHEUS SERVER

Prometheus scrapes metrics from exporters at regular intervals (e.g., every 15s).

4 STORAGE (TSDB)

Metrics are stored in the local Time Series Database (TSDB) with timestamps and labels.

5 PROMQL

PromQL queries the data in TSDB to retrieve, aggregate and analyze metrics.

6 GRAFANA

Grafana visualizes metrics with dashboards, panels, and explorations.

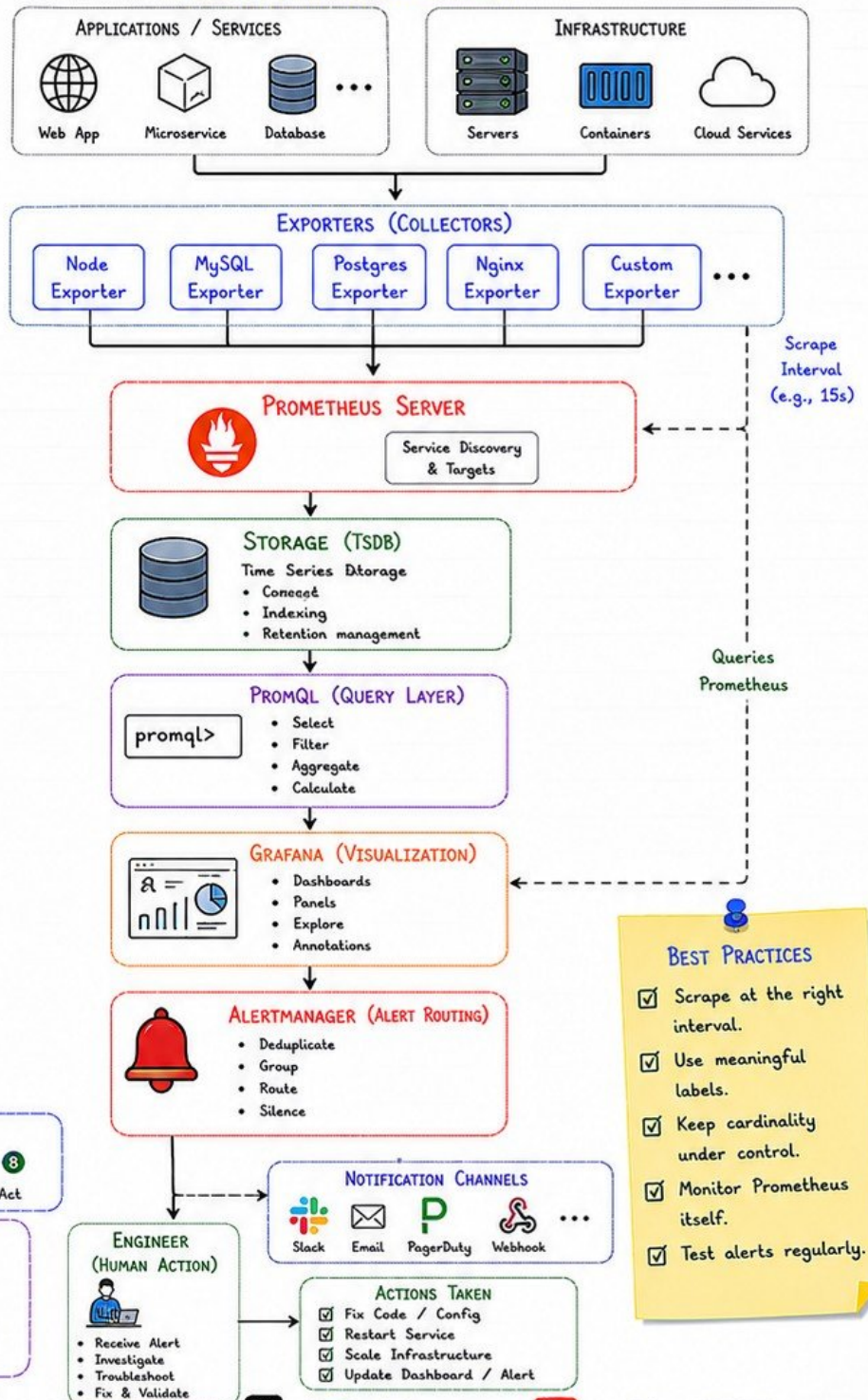
7 ALERTMANAGER

Alertmanager handles alerts from Prometheus, deduplicates, groups, and routes notifications.

8 ENGINEER

Engineers receive alerts, investigate issues, and take action to resolve problems.

PRODUCTION WORKFLOW DIAGRAM



DATA FLOW SUMMARY

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8
Collect → Store → Query → Visualize → Alert → Act

PRODUCTION NOTES

- Prometheus is pull-based (scrapes metrics).
- Exporters expose metrics in /metrics endpoint.
- Data is stored with labels and timestamps.
- PromQL is powerful and expressive.
- Alerts should be actionable and meaningful.
- Good dashboards + alerts = good observability.

BEST PRACTICES

- ✓ Scrape at the right interval.
- ✓ Use meaningful labels.
- ✓ Keep cardinality under control.
- ✓ Monitor Prometheus itself.
- ✓ Test alerts regularly.

**REMEMBER**

Start small.
One server.
Big learning.
Real impact!

18. FIRST MONITORING PROJECT

HANDS-ON PROJECT

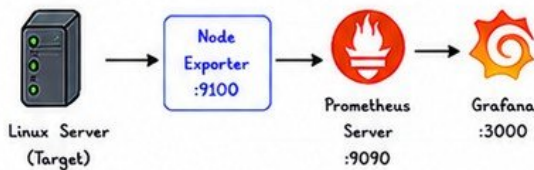
In this project, we will monitor **one Linux server** using Prometheus and visualize key system metrics. We will collect **CPU, Memory, Disk and Network** metrics.

PROJECT GOAL



- ✓ Set up monitoring for 1 Linux server.
- ✓ Collect system metrics.
- ✓ Query metrics with PromQL.
- ✓ Visualize in Grafana.
- ✓ Understand the full workflow.

1 ARCHITECTURE



2 SETUP OVERVIEW

- ✓ Install Node Exporter on the Linux server.
- ✓ Install Prometheus on a monitoring server.
- ✓ Configure Prometheus to scrape Node Exporter.
- ✓ Verify the target is up.
- ✓ Run PromQL queries.
- ✓ Visualize in Grafana.

3 COMPONENTS & PORTS

COMPONENT	PORT
Node Exporter	9100
Prometheus	9090
Grafana	3000

4 COLLECT (WHAT WE WILL MONITOR)



CPU

- Cpu usage, load average, cpu cores.



MEMORY

- Total, used, available, used %.



DISK

- Filesystem usage, disk read/write, IO.



NETWORK

- Network traffic in/out, packets, errors.

5 CONFIGURATION

A. NODE EXPORTER (ON LINUX SERVER)

```
# Download and run
wget https://github.com/prometheus/node_exporter/releases/latest/download/node_exporter-<version>.linux-amd64.tar.gz
tar -xzf node_exporter-<version>.linux-amd64.tar.gz
cd node_exporter-<version>.linux-amd64
./node_exporter
```

Verify: `curl http://<server-ip>:9100/metrics`

B. PROMETHEUS (PROMETHEUS.YML)

```
scrape_configs:
  - job_name: 'linux-server'
    static_configs:
      - targets: ['<server-ip>:9100']
```

Reload Prometheus and check targets.

6 VERIFY TARGETS

Go to Prometheus UI:

`http://<prometheus-ip>:9090/targets`

Expected Output:

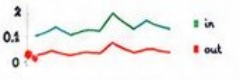
Targets

All Unhealthy Collapse All

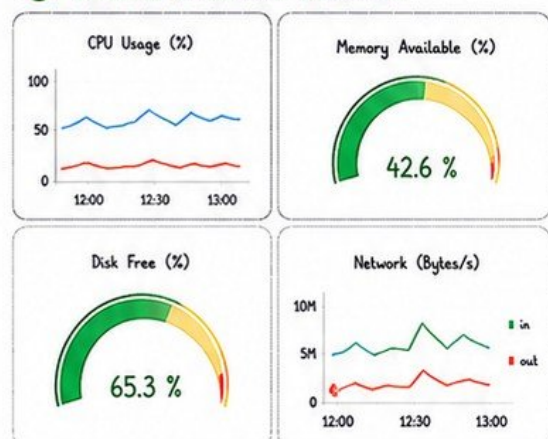
linux-server (1/1 up)

Endpoint	State	Labels	Last Scrape	Scrape	Duration	Error
http://<server-ip>:9100/metrics	UP	job="linux-server" instance="<server-ip>:9100"	5s ago	15s	12ms	

7 RUN YOUR FIRST PROMQL QUERIES

QUERY	DESCRIPTION	EXPECTED OUTPUT (EXAMPLE)	GRAFANA PANEL TYPE
up	Check if the target is up.	1 (up) or 0 (down)	Stat
node_cpu_seconds_total rate(node_cpu_seconds_total[5m])	CPU usage by mode.		Time series (Line)
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100	Memory available %.	42.6 %	Gauge
node_filesystem_avail_bytes {fstype!~"tmpfs overlay"} / node_filesystem_size_bytes {fstype!~"tmpfs overlay"} * 100	Disk free %.		Gauge
rate(node_network_receive_bytes_total[5m]) rate(node_network_transmit_bytes_total[5m])	Network in/out (bytes/sec).		Time series (Line)

8 EXPECTED DASHBOARD (GRAFANA)



9 PROJECT CHECKLIST

- ✓ Node Exporter is running and accessible.
- ✓ Prometheus is scraping the target.
- ✓ Target status is UP.
- ✓ Metrics are being collected.
- ✓ PromQL queries return results.
- ✓ Grafana dashboard is working.
- ✓ Alerts can be created (next step!).

CONGRATULATIONS!



You just completed your first monitoring project!

You are now collecting real metrics from a real server.



TIP

Always start with one host.
Master the basics before
scaling to multiple systems!

REMEMBER

Mistakes are normal. Learning from them early saves you from production disasters. Don't skip the basics!

19. COMMON BEGINNER MISTAKES

Everyone makes mistakes when starting with Prometheus. Here are the most common ones I see in production environments – with real examples and how to avoid them.

BIG IDEA

Good monitoring is not about collecting the most metrics. It's about collecting the right metrics and taking action.

MISTAKE	EXAMPLE	PRODUCTION ADVICE
1 MONITORING EVERYTHING 	- Scraping hundreds of useless metrics (debug, verbose, or low-value metrics). - Leads to high storage usage and slow queries. <pre>scrape_interval: 15s # for everything</pre>	 Follow a "monitor less, but better" approach. Focus on SLI/SLOs and business-relevant metrics. Start small and expand gradually.
2 WRONG SCRAPE INTERVAL 	- Using too low interval (e.g., 5s) for everything. - Causes high CPU, network, and storage cost. <pre>scrape_interval: 5s</pre>	 Use appropriate intervals: 15s – 30s for standard metrics 1m – 5m for low-frequency metrics - Understand metric volatility.
3 HIGH CARDINALITY (LABEL EXPLOSION) 	- Using labels like user_id, session_id, ip, or request_id. - Millions of time series = high cost & slow queries. <pre>http_requests_total{user_id="12345"}</pre>	 Keep labels low-cardinality. Use labels for context, not identity. Avoid unbounded values.
4 MISSING EXPORTERS 	- Important components not being exported. - You can't monitor what you don't collect. <pre>No exporter for MySQL, Redis, Nginx</pre>	 Identify all critical components. Install and configure the right exporters. Verify targets regularly.
5 IGNORING ALERTS 	- Alerts firing but no one responds. - Leads to alert fatigue and real issues being missed. <pre>ALERT DiskFull IF node_filesystem_avail_bytes < 1G</pre>	 Treat alerts like production incidents. Route alerts to the right people. Review and tune alert rules.
6 NOT TESTING CONFIGURATIONS 	- Pushing config changes without validation. - Syntax errors break scraping and alerts. <pre>prometheus.yml (syntax error)</pre>	 Always validate before reload: <pre>promtool check config prometheus.yml</pre> Test in a staging environment first.
7 POOR NAMING CONVENTIONS 	- Inconsistent or unclear metric/label names. - Makes queries hard and dashboards messy. <pre>cpususage vs cpu_usage vs CPU_Usage</pre>	 Follow Prometheus naming best practices: lowercase, underscores, units. Be consistent across the organization.



PRODUCTION ADVICE – THE GOLDEN RULES

- ✓ Monitor what matters to your business.
- ✓ Understand your infrastructure and applications.
- ✓ Test configurations, alerts, and dashboards regularly.
- ✓ Review metrics, labels, and costs continuously.
- ✓ Automate everything (infrastructure + monitoring).
- ✓ Document your monitoring strategy.
- ✓ Keep learning and stay curious!



KEY TAKEAWAY

Good monitoring is not about tools. It's about good decisions and continuous improvement. Start small, learn, and build with purpose.



Everyone starts somewhere. The difference is the ones who keep learning and keep building!

You've got this! 🍀



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

REMEMBER

You don't need to know everything. You need to understand deeply and apply consistently.

20. VOLUME 1 REVIEW & MINI PROJECT



GREAT WORK!

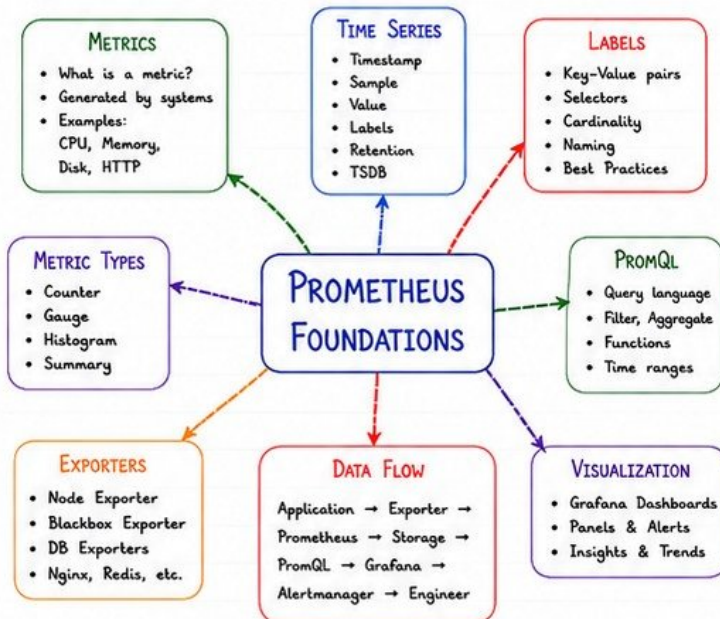
You've learned the foundations of Prometheus. You now understand how metrics flow from systems to insights. Keep practicing!



★ WHAT WE COVERED

- ☒ Why metrics matter
- ☒ How Prometheus works
- ☒ Metrics, time series & types
- ☒ Labels & best practices
- ☒ Exporters & data flow
- ☒ First monitoring project

1 QUICK REVISION MIND MAP



2 MINI PROJECT

Build a basic monitoring stack using:



Prometheus

+



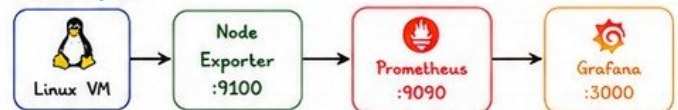
Node Exporter

+



One Linux VM

ARCHITECTURE



- 1 Set up 1 Linux VM (Ubuntu/CentOS)
- 2 Install and run Node Exporter
- 3 Install and run Prometheus
- 4 Configure Prometheus to scrape Node Exporter
- 5 Access Prometheus UI and verify target
- 6 Run basic PromQL queries
- 7 Visualize metrics in Grafana

SUCCESS CRITERIA



- ☒ Target "node" is UP in Prometheus
- ☒ CPU, Memory, Disk, Network metrics are visible
- ☒ Dashboards show real-time data

3 VERIFICATION CHECKLIST

- ☒ Node Exporter service is running
- ☒ Prometheus service is running
- ☒ Targets page shows node as UP
- ☒ Metrics endpoint is accessible (/metrics)
- ☒ Basic metrics are being scraped
- ☒ PromQL queries return results
- ☒ Grafana can connect to Prometheus
- ☒ Dashboards display metrics correctly



4 KEY CONCEPTS RECAP

- Metrics are time-based measurements
- Prometheus pulls metrics from targets
- Data is stored as time series in TSDB
- Labels add context and power queries
- PromQL is used to query and analyze data
- Grafana visualizes and explores metrics
- Alertmanager handles alerts & routing
- Everything works together as a pipeline

5 INTERVIEW QUESTIONS ?

- 1 What is Prometheus and how does it work?
- 2 What is the difference between Counter and Gauge?
- 3 How does Prometheus store data?
- 4 What are labels and why are they important?
- 5 What is cardinality and why does it matter?
- 6 How do you write a PromQL query to find top CPU usage by instance?
- 7 How do you add a new exporter to Prometheus?



PRODUCTION READINESS CHECKLIST

- ☒ Use appropriate scrape intervals
- ☒ Monitor Prometheus health (self-monitoring)
- ☒ Use meaningful labels and low cardinality
- ☒ Configure recording rules for heavy queries
- ☒ Set up Alertmanager with proper routing
- ☒ Back up Prometheus data (TSDB)
- ☒ Keep retention and resource limits in check
- ☒ Dashboards follow best practices
- ☒ Test alerts and dashboards regularly

7 NEXT VOLUME PREVIEW

Prometheus Handbook

Volume 2:

PromQL, Alerting & Production Monitoring

- ☒ Deep dive into PromQL
- ☒ Alerting rules & best practices
- ☒ Alertmanager in production
- ☒ Dashboards & observability patterns
- ☒ Real-world monitoring strategies



“ The best way to master Prometheus is to build, break, fix, and repeat. Keep building. You're doing great! ”

STAY CONNECTED



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

THANK YOU FOR GOING THROUGH VOLUME 1!

Keep learning, keep building, keep shipping! ❤️

Shared by convenient and free Image to PDF
Converter: <https://st.simplesign.itd/7ziEng>